

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

11. A-D átalakítás

Írta: Trádler Máté

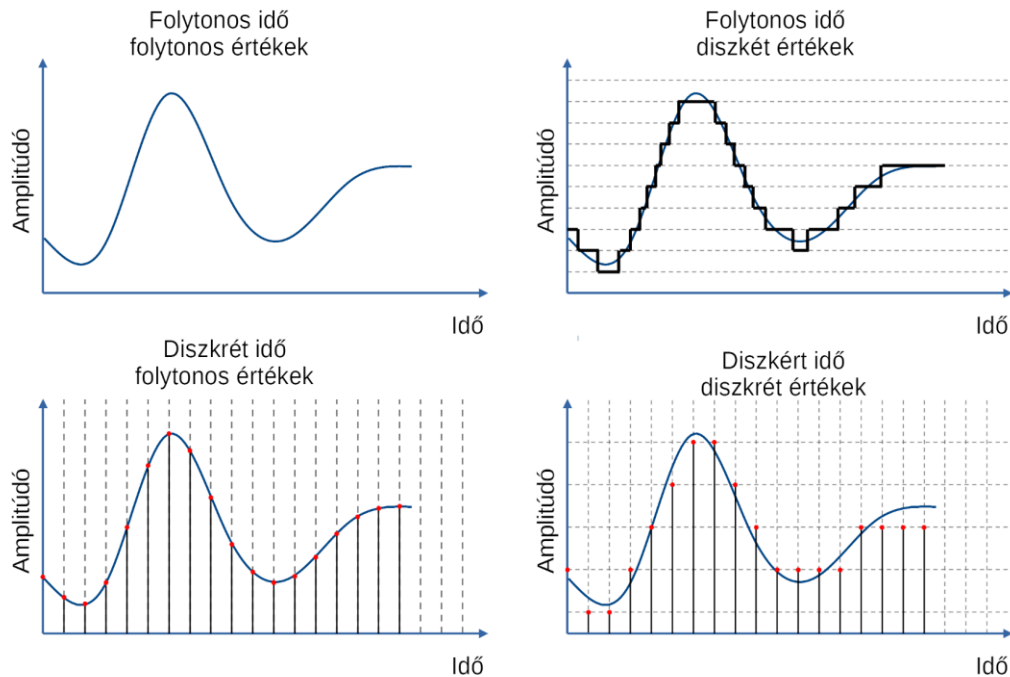
Lektorálta: Horváth Tamás

Ebben a tananyagrészben az STM32-es mikrokontroller analóg-digitális átalakítójával (későbbiekben ADC - Analog Digital Converter) fogunk megbarátkozni. Először nagy vonalakban megnézzük, hogy mit is érdemes tudni az ADC-ről (ha szeretnél még róla általánosan olvasni, akkor ajánlom figyelmedbe a Kristálytisza Elektronika tananyagának 22. részét), ezt követően megismerkedünk a fejlesztőpanelen lévő vezérlő ADC moduljának beállítási lehetőségeivel, majd érdekes feladatokon keresztül elsajátítjuk annak használatát.

ÁTTEKINTÉS, AVAGY MI IS AZ AZ ANALÓG-DIGITÁLIS KONVERTER?

Először nézzük meg, hogy mikor is van szükségünk egyáltalán ADC-re, milyen feladatot képes ellátni? Sokféle analóg jelet mérhetünk: hőmérsékletet, távolságot, fényerősséget, feszültséget, áramot stb., de ezeket először egyetemesen feszültségjellé kell alakítanunk, hogy az ADC fel tudja dolgozni (ezt az átalakítást végzik a szenzorok, melyekről bővebben a 13. tananyagrészben olvashattok majd). Jó példa továbbá a különböző áramkörökön lévő tápegységek feszültségének visszamérése, mivel ez által megbizonyosodhatunk annak működőképességéről valamint beavatkozhatunk egy esetleges hiba esetén. A mérni kívánt feszültség egy analóg jel, azaz minden időpillanatban jól meghatározott értékkel rendelkezik és végtelen sok különböző értéket vehet fel. Ezzel ellentétben a digitális jelek adott időközönként változtatják csak az értéküket, azaz nem lehet csak meghatározottakat, ezt idegen szóval úgy mondjuk, hogy a jel kvantált. Ilyen digitális jelre jó példa egy órajel, ami fix időközönként (amely a periódusidő fele) változtatja az értékét két jól meghatározott feszültségszint között.

A mikroprocesszorok csak és kizárólag digitális jelekkel tudnak dolgozni, a folytonos jeleket digitális jelekké kell alakítani, hogy értelmezni tudják őket. A következő ábrán egy analóg jel és annak diszkrétizált alakjai láthatók.

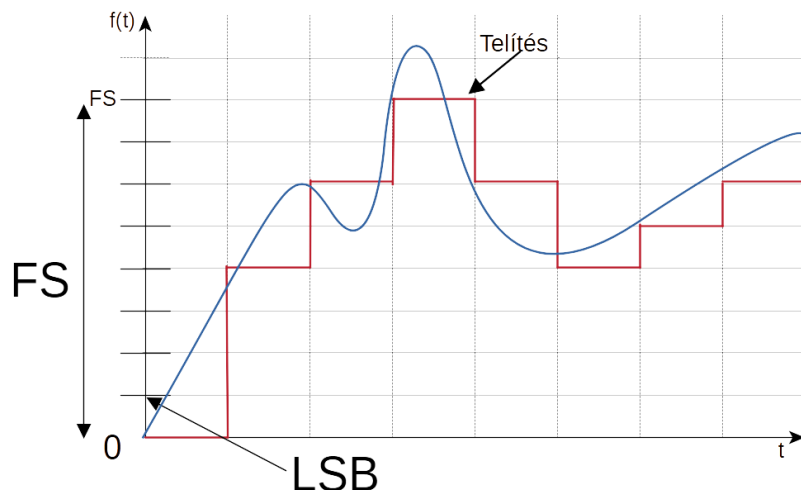


1. ábra: Különböző típusú jelek függvényei

A folytonos jelet egy meghatározott hosszúságú bináris számsorozattal fogjuk leírni és az ehhez szükséges átalakítást fogja elvégezni nekünk az A/D (Analóg/Digitális) átalakító. Mint mindennek, sajnos az ADC-knek is vannak korlátai. Mivel egy végtelen sok értéket tartalmazó halmazt szeretnénk egy véges számosságú halmazba leképezni, így elkerülhetetlen az információvesztés. Az analóg-digitális átalakítás jellemezhető a felbontásával, ami azt mutatja meg, hogy összesen hány darab különböző értéket tud felvenni a diszkrét jel. Ezt az értéket 2 hatványaként szoktuk kifejezni, mert biteken, bináris formában ábrázoljuk az átalakított értékeket. Vegyük példának azt, hogy az A/D átalakító 10 bites. Ebben az esetben a felbontása 10 bit, tehát $2^{10} = 1\,024$ különböző értéket képes előállítani a kimenetén, melynek maximális értéke 1 023, a minimális pedig 0. Most már tudjuk, hogy egy fizikai mennyiségből származtatott feszültségértéket milyen pontosan írunk le, de nem mindegy, hogy ezt a felbontást 0 - 230 V vagy 0 - 1 V között alkalmazzuk. Ennek a jellemzésére szolgál a kivezérlési tartomány (angolul Full Scale). Ez jelenti azt a bemeneti feszültségtartományt, melyen belül a kimenet a bemenet lineáris függvénye. Ha a bemenetre olyan jel kerül, amely kívül esik a tartományon, akkor az úgynevezett telítődés jelenségét figyelhetjük meg, miszerint az ADC nem tud elég nagy kimenetet produkálni. Ilyen esetben hiába nő a bemenet, a kimenet csak a maximumot fogja mutatni, vagy ellenkező esetben hiába csökken a bemeneti feszültség, a konverter nem tud nullánál kisebb kimenetet előállítani. Általában ez a tartomány 0 V-tól kezdődik és az ADC referenciafeszültségénél ér véget. Azt a mérőszámot, ami a teljes kivezérlési tartomány és a felbontás arányát fejezi ki, LSB-nek nevezzük (Least Significant Bit). Ez mondja meg nekünk, hogy két szomszédos, bináris számértékkel reprezentált feszültség szint között mekkora különbség van.

Az adott bináris értéket összeszorozva az LSB-vel megkapjuk a mért feszültségértéket. Fontos megjegyezni, hogy ez nem azt mondja meg, hogy mekkora feszültség volt a bemeneten, hanem azt, hogy mit reprezentál a bináris érték. A különbség oka a korábban említett kvantálásban keresendő.

Az ábrázolás megértése után térjünk rá arra, hogy hogyan is alakítjuk át a jelet. A válasz mintavétellel, kvantálással, majd digitális érték hozzárendelésével. Ez a folyamat úgy zajlik, hogy adott időközönként megvizsgáljuk a folytonos jel értékét, majd megnézzük, hogy az melyik bináris reprezentációhoz áll legközelebb és azt választjuk értékül. Nevezhetjük lépcsőfokoknak a bináris számértékeket, mert két ilyen lépcsőfok között minden minta vagy felfelé, vagy lefelé kerekítődik.



2. ábra: Folytonos jel átalakítása

Kitekintés:

Számoljunk egy kicsit! Vegyünk egy 3,3 V-os referenciafeszültséggel ellátott 12 bites ADC-t. Határozzuk meg az átalakítóhoz tartozó főbb jellemzőket.

1. Milyen feszültségtartományban lesz érvényes az átalakított jelünk?
2. Mekkora lesz a felbontás?
3. Mi az a legkisebb érték, amit meg tudunk majd különböztetni, vagyis mekkora lesz az LSB értéke?

Válaszok:

1. 0 V - 3,3 V közötti tartományon belül lesznek érvényesek az átalakított jelek.
2. A felbontás meghatározásához ismernünk kell, hogy hány biten történik az átalakítás. Ez a példánkban 12 bit. Ebből következik, hogy 2^{12} , vagyis 4 096 értéket tudunk megkülönböztetni.
3. LSB számításánál tudnunk kell, hogy milyen széles a teljes kivezérelhetőségi tartomány (legnagyobb és legkisebb feszültségérték közti különbség), ami most $3,3 \text{ V} - 0 \text{ V} = 3,3 \text{ V}$. Másik fontos paraméter a felbontás, ami az előzőek alapján 4 096.

Így már könnyen megkapjuk az LSB értékét a következő formula alapján:

$$\text{mérhető tartomány} / (\text{felbontás} - 1),$$

vagyis:

$$3,3 \text{ V} / 4\,095 = 0,000\,805\,860\,8 \text{ V} (0,805\,860\,8 \text{ mV}).$$

Az osztóban szereplő kifejezés, amelynek értéke jelen esetben 4 095 (felbontás - 1), amely azt

jelenti, hogy hány darab kis egyenlő részre osztódik fel a tartomány. Ez onnan adódik, hogy az utolsó bináris kódhoz (ami 12 darab 1-et tartalmaz), tartozik a legnagyobb érték a tartományon. Ennek a bináris számnak a decimális értéke 4 095.

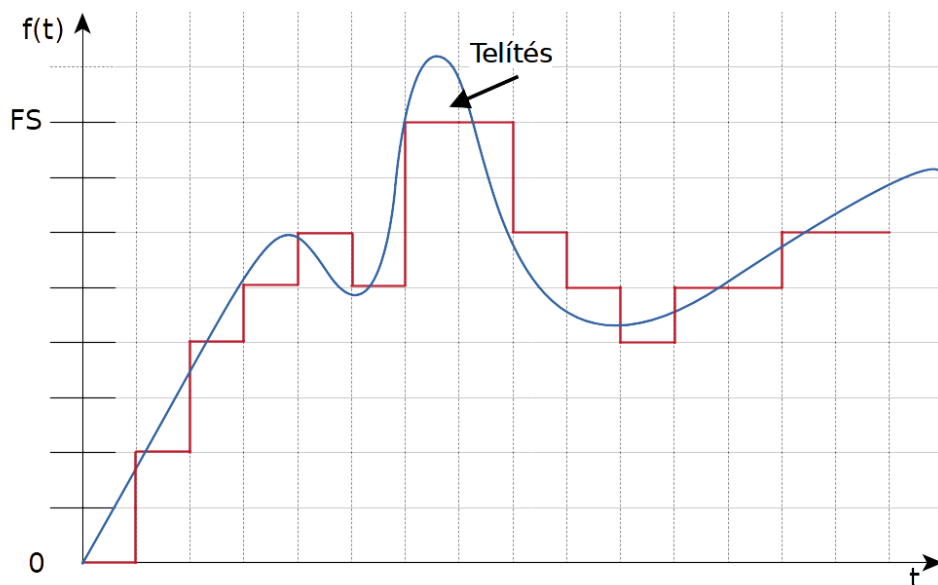
A kapott LSB érték sok tizedesjegyből áll. Nézzük meg, hogy mekkora hibát kapunk különböző kerekítések mellett! A 0,805 860 8 mV két tizedesjegyre való kerekítésekor 0,81 mV-ot kapunk. A legnagyobb kerekítésből származó hiba meghatározásához ki kell számolnunk a kerekítéssel adódó tartományunk tetejét. Ezt az előző formula átrendezésével meg tudjuk tenni:

$$\text{LSB} * (\text{felbontás} - 1)$$

$$, \text{ vagyis } 0,81 \text{ mV} * 4\,095 = 3\,316,95 \text{ mV}.$$

Ez 16,95 mV-nyi kerekítési hibát jelent (3 316,95 mV - 3 300 mV-ból adódóan), ami LSB-ben kifejezve több, mint 21 LSB-nyi hibát jelentene. Ez egy jelentős kerekítési hiba. Számoljunk hasonló módom három tizedesjegyre történő kerekítés esetén is! Ekkor a kerekített LSB értékünk 0,806 mV. A kerekítéssel kapott tartományunk teteje 3 300,57 mV, amely 0,57 mV-nyi kerekítési hibát jelent. Ez kevesebb, mint 1 LSB-nyi hiba, amely az esetek nagy részében már megengedhető.

Vessünk még egy pillantást az előző ábrára! Láthatjuk, hogy a folytonos jelből vett minták nem írják le megfelelően az eredeti jelet (ez persze egy árnyalt kifejezés, mivel minden esetben a tervező feladata annak meghatározása, hogy mekkora hiba alatt tekinti megfelelőnek az eredeti jelet). Látható, hogy az átalakítás nem alakhű, mivel gyorsabban változik a jel, mint ahogy mi mintát veszünk belőle. Ennek javítása érdekében többször kell mintát vennünk a jelből, azaz növelnünk kell a mintavételünk gyakoriságát, frekvenciáját, ezáltal pontosabban leírhatjuk a bemeneti jelet. Nézzük meg az előző jelünket nagyobb mintavételi frekvencia esetén! Látható, hogy a frekvencia növelésével a digitális jel jobb reprezentációját adja az analóg jelnek, mint az előző esetben.



3. ábra: Folytonos jel átalakítása sűrűbb mintavételezéssel

Kitekintés:

Mi történik, ha a kivezérlési tartományon kívül esik az ADC-n lévő feszültség? Ekkor az ADC telítésbe megy, ami azt jelenti, hogy elérte a mérési tartományának a tetejét, vagyis a legnagyobb átalakítható értéket. Például, ha a referenciafeszültség (ami egyben az átalakítás felső határa) 3,3 V és az ADC bemenetére 3,5 V kerül, akkor szintén a 3,3 V-nak megfelelő digitális értéket fogjuk látni az ADC kimenetén. Természetesen akármekkora feszültség nem kerülhet az ADC bemenetére annak károsodása nélkül, (a meghibásodás elkerülése érdekében) a maximális feszültségérték a fejlesztőpanelen található mikrokontroller esetében 4 V. A mikrovezérlőnk adatlapjában találhatjuk meg ezt az értéket, az abszolút maximális értékek alatt (5.2 Absolute maximum ratings). Azonban a funkcionális működés csak 3,6 V-ig garantált, eddig az értékig biztosan telítésbe fog menni az ADC konverterünk, ezt az értéket az általános működési feltételek alatt olvashatjuk ki a mikrovezérlő adatlapjából (5.3.1 General operating conditions). Ezzel a jelenséggel jó, ha tisztában vagyunk, de általában elhanyagolható, mivel a mérési tartományt a telítési határon belülre választjuk.

MIT IS REJT MAGÁBAN AZ STM32-ES ADC MODUL?

A következőkben megismerkedünk az STM32F4-es család ADC perifériájával. A mikrovezérlőbe három darab 12 bites szukcesszív approximációs ADC-t (amelyhez rövid leírás a lentebb található kitekintésben olvasható) integráltak, amelyek mindegyikéhez 16 külső csatorna kapcsolódik. Ez azt jelenti, hogy egy ADC periféria maximum 16 különböző bemenetre érkező analóg jelet tud feldolgozni. A teljes kivezérlési tartományra tekintve azt láthatjuk, hogy 0 V-tól egészen a referenciafeszültségig (későbbiekben VREF, amely az angol Voltage REference rövidítéséből származik) képes átalakításokat végezni, ezt a feszültséget a mikrovezérlő dedikált VREF lábára kell kapcsolni. Erre a feszültségszintre vonatkozó követelmény, hogy legalább 1,8 V és legfeljebb a tápfeszültséggel megegyező értékű lehet. Az ADC használható azonban belső referenciafeszültséggel is, aminek nagysága a mikrovezérlő tápfeszültségével egyezik meg, ennek értéke a fejlesztőpanel tekintetében 3,3 V. Külső referenciafeszültség használatával a teljes kivezérlési tartományt szűkíthetjük, ezzel növelve az átalakítás érzékenységet, így kisebb feszültségértékbeli különbséget is meg tudunk megkülönböztetni. További előnye még, hogy stabilabb és pontosabb feszültségforrás csatlakoztatására nyílik mód egy különálló precíziós tápforrás felhasználásával. Természetesen ez a változtatás hátrányos is lehet, például a szűkített mérési tartományon a zajok sokkal nagyobb mértékben jelennek meg.

KONVERZIÓS MÓDOK

Négyféle konverziós mód közül választhatunk:

- egyszeri (Single)
- folyamatos (Continuous)
- szkenneléses (Scan)
- szakaszos (Discontinuous)

Egyszeri mód (angolul Single Conversion Mode) a legegyszerűbb, mivel ekkor az átalakítást az éppen aktuális analóg értékre végezzük el, majd pedig leáll a folyamat. Ezt a módot akkor célszerű használni, amikor a szoftver futása során egy dedikált időpillanatban szeretnénk átalakítást végezni.

Folyamatos mód esetében (angolul Continuous Conversion Mode), amikor befejeződött az átalakítás, kezdődik is a következő, azaz folyamatos a konverzió. Ezt a módot használhatjuk, amikor például egy hőmérséklet szenzor értékét folyamatosan szeretnénk mérni.

Szkenneléses mód (angolul Scan Mode) esetén lényegében egyszeri vagy folyamatos átalakítás zajlik le, csak nem egy, hanem több csatornára. A konverzió végét az utolsó átalakítást követően kapjuk. Ezt a módot érdemes választani, amikor például több szenzor kimenetét szeretnénk folyamatosan mérni.

A szakaszos mód (angolul Discontinuous Mode) pedig azt takarja, hogy egy megszakítás hatására fog csak elindulni az átalakítás, ilyen lehet például egy számláló lejárt. Kézenfekvő választás ez a mód, ha például egy eszköz akkumulátorának feszültségét szeretnénk mérni bizonyos időközönként.

Az átalakítás ideje több paraméterből tevődik össze. Függ a választott bitszélességtől, az alkalmazott ADC órajel-frekvenciájától (ami a fejlesztőpanelen használt mikrokontroller esetében maximálisan 42 MHz lehet) és a mintavétel időtartamától. Nézzünk meg erre egy szemléltető példát! A jelünket 12 biten szeretnénk digitalizálni, 3 ADC órajelciklusig tartó mintavétel mellett. Ez 15 ADC órajelciklus alatt fog lezajlani, amely a maximális, 42 MHz-es órajel frekvencia mellett $2,8 \cdot 10^6$ -on (periódusidő: $1/42000000$ s, egy konvertálás ideje = 15 periódus) elvégzett konverziót jelent másodpercenként, vagyis 2,8 Msps-t. A mintavételi sebességet Msps-ben szokás megadni, amely a mintavételek számát jelenti másodpercenként (Mega samples per second), a mega előtag pedig a milliós nagyságrendre utal.

Kitekintés:

Fontos a megfelelő ADC konverziósebesség és felbontás választása. Nem célszerű ágyúval lőni verébre, mert erőforrást kötünk le úgymond feleslegesen. Például, ha másodpercenként kell mintavételeznünk egy jelet tizedpontosággal, akkor nem célszerű a leggyorsabb és legnagyobb felbontással mintavételezni azt.

KIOLVASÁSI METÓDUSOK

Nézzük meg milyen lehetőségeink adódnak az átalakítás eredményének kiolvasására. Három különböző módon tehetjük meg ezt, amelyek a következők:

- blokkoló (Polling)
- megszakításos (Interrupt)
- DMA (Direct Memory Access) - közvetlen memória-hozzáférés

A legegyszerűbb a blokkoló mód (angolul Polling mode), ez azt jelenti, hogy a konverzió indítását követően addig várunk az eredményre, amíg az el nem készült számunkra. Ezáltal a mikrovezérlőnket lefoglaljuk a várakozás során, nem fog más számítást elvégezni ez idő alatt. Használhatjuk az ADC perifériánkat megszakításos (angolul Interrupt) módban is. Ekkor a mikrovezérlőben egy megszakítás generálódik a konverzió befejeztekor, majd lefut a hozzá tartozó rövid, néhány soros függvény (angolul callback függvénynek nevezik ezeket). A függvény lefutása a programunkat megszakítva történik meg. Ebben az esetben a konverzió alatt a mikrovezérlőnk nincsen lefoglalva, akár más műveletet is tud közben végezni, mialatt a konverzió zajlik a háttérben. Harmadik módja a digitalizált eredmény kiolvasásának a DMA-s mód. Ennek során sok processzoridőt spórolhatjuk meg, mivel egy külön egység

fogja kezelni és eltárolni a konverzió eredményeit (közvetlen memória-hozzáféréssel), így levéve a terhet a processzorról, a feldolgozás pedig egy általunk definiált programrészen történik meg. Az ADC modulon belül található egy analóg watchdog-nak elnevezett, igen hasznos funkció. De mire is szolgál ez? Előfordulhat, hogy egy jel csak bizonyos feszültségtartományon belül megfelelő számunkra, például egy hőmérsékletszenzor esetében. Ha ez a jel elhagyja a tartományt, arról mi egyből szeretnénk értesítést kapni. Ebben tud segítséget nyújtani az analóg watchdog, ami minden értéket összehasonlít a határértékekkel, mint egy ablakkomparátor (lásd a Kristálytisza Elektronika tananyagának 14. része). A tartomány alsó és felső határát kell meghatároznunk, majd ezek után, ha a mért érték az általunk specifikált tartományon kívülre kerül, akkor erről értesítést fogunk kapni. A megfigyelés a háttérben zajlik, így nekünk szoftveresen nem kell figyelnünk az eseményt, ezáltal könnyítve meg dolgunkat.

Kitekintés:

Az egyik legelterjedtebb átalakítótípus a szukcesszív approximációs ADC. A mikrovezérlőkbe előszeretettel integrálják ezt pontossága, gyorsasága és alacsony költsége miatt. Működése során alulról közelíti az adott értéket és elve a folyamatos megközelítésen alapszik. Ez azt jelenti, hogy a legnagyobbtól a legkisebb helyiértékű bitig haladva folyamatosan összehasonlításra kerül a konvertálni kívánt jel egy előállított analóg jellel. Az adott helyiértéknek megfelelő analóg jelet egy digitális-analóg konverter (röviden DAC) fogja előállítani. A DAC-t részletesen a következő tananyagrészben mutatjuk be. Érdeemes egy számkereső játékként gondolni erre az ADC átalakításra: (10 bites ADC esetén)

Jel: gondoltam egy számra 0 és 1 023 között (ez a szám az 583,78).

ADC: nagyobb mint 512?

Jel: igen (1)

ADC: nagyobb mint $512 + 256 = 768$?

Jel: nem (0)

ADC: nagyobb mint $512 + 128 = 640$?

Jel: nem (0)

ADC: nagyobb mint $512 + 64 = 576$?

Jel: igen (1)

ADC nagyobb mint $576 + 32 = 608$?

Jel: nem (0)

ADC nagyobb mint $576 + 16 = 592$?

Jel: nem (0)

ADC nagyobb mint $576 + 8 = 584$?

Jel: nem (0)

ADC nagyobb mint $576 + 4 = 580$?

Jel: igen (1)

ADC nagyobb mint $580+2 = 582$?

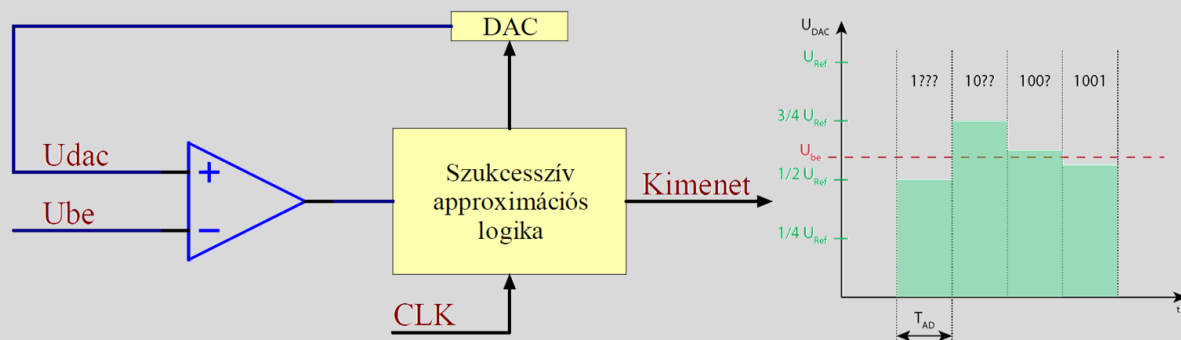
Jel: igen (1)

ADC nagyobb mint $582+1 = 583$?

Jel: igen (1)

Az átalakított jel bináris reprezentációja: 1001000111 -> 583

Lényegében minden helyiérték-vizsgálatnál az ADC felezi a tartományt. Mivel helyiértékenként történik az összehasonlítás, így ahány biten történik az átalakítás, annyi órajelciklus alatt fog lezajlani. A következő ábra ezt szemlélteti.



4. ábra: Szukcesszív approximációs ADC felépítése és működése

A bemutatott ADC típus mellett más működési elven alapuló ADC-k is léteznek, mint például a Flash ADC, Szigma-delta átalakító vagy a Dual slope átalakító. Ha szeretnéd bővebben is megismerni ezeket a típusokat, akkor a Kristálytisza Elektronika tananyagának 22. részében megtalálod az említett típusokat, működésük részletes leírásával együtt.

AZ ADC PERIFÉRIA REGISZTEREI:

Látni fogjuk, hogy milyen egyszerűen kezelhető az ADC az ST által biztosított könyvtárral, mivel elrejtjük előlünk a regiszterkezelést. Ez meggyorsítja és megkönnyíti számunkra a periféria használatát és így a fejlesztést, illetve a szoftverünk hordozhatóságát is segíti különböző STM32-es mikrokontroller-típusok között. Mielőtt rátérnénk a könyvtár használatára, érdemes nagyvonalakban megismernünk az STM32 ADC perifériájához tartozó fontosabb regisztereket, amelyek a folyamatok mélyebb megértésében segíthetnek.

Első és egyik legfontosabb regiszter számunkra az ADC_SR regiszter, azaz a státusz regiszter. Innen számos fontos információt nyerhetünk perifériánk aktuális állapotáról, mint például, hogy elindult-e vagy befejeződött-e már az átalakítás, továbbá itt regisztrálódik még a watchdog esemény is.

Két vezérlőregisztert is találhatunk ADC_CR1 és ADC_CR2 néven. Ezek a regiszterek tartalmazzák a periféria konfigurációját, melyet inicializálás során adunk meg és menet közben is módosíthatunk igényeinknek megfelelően. Mint például hány bit szélességgel, milyen mintavételi frekvenciával és milyen módban szeretnénk az ADC-t használni. Itt lehet továbbá a különböző megszakítások engedélyezését is beállítani.

Az ADC_SMPR1 és ADC_SMPR2 regiszterekben adhatjuk meg azt, hogy egy konverzió során hány órajelciklusig mintavételezzünk a jeltől. Természetesen, ha több ADC órajelcikluson keresztül veszünk mintát, akkor az pontosabb eredménnyel szolgálhat, de persze hosszabb ideig is tart.

A korábban már említett analóg watchdog határértékeit (azaz, hogy mikor jelezzen) az ADC_HTR (felső határérték) és az ADC_LTR (alsó határérték) regiszterekkel tudjuk beállítani.

Utoljára maradt az ADC_DR regiszter, ahová a konverzió eredménye kerül. Innen fogjuk majd kinyerni a az analóg jel bináris reprezentációját.

A következő táblázat összefoglalja az előzőekben ismertetett regisztereket.

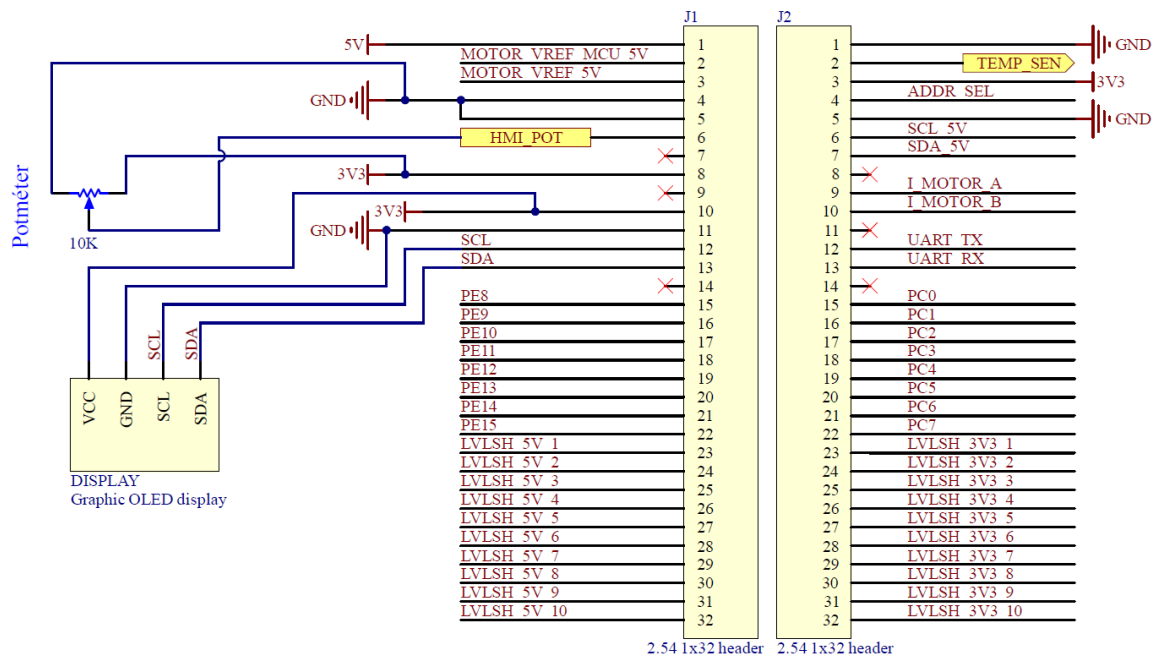
Regiszter neve:	Rövidített neve:	Funkciója:
Státuszregiszter	ADC_SR	ADC aktuális állapotát reprezentálja
Vezérlőregiszterek	ADC_CR1 és ADC_CR2	ADC konfigurációját tartalmazza
Mintavételezési regiszterek	ADC_SMPR1 és ADC_SMPR2	Csatornák mintavételezésére vonatkozó beállításokat tartalmazza
Adatregiszter	ADC_DR	A végrehajtott konverzió eredményét tartalmazó regiszter
Watchdog felső határérték regisztere	ADC_HTR	Analóg watchdog felső határértékét tároló regiszter
Watchdog alsó határérték regisztere	ADC_LTR	Analóg watchdog alsó határértékét tároló regiszter

1. táblázat: Az ADC regiszterei

FELADAT

A KAPCSOLÁS ÖSSZEÁLLÍTÁSA:

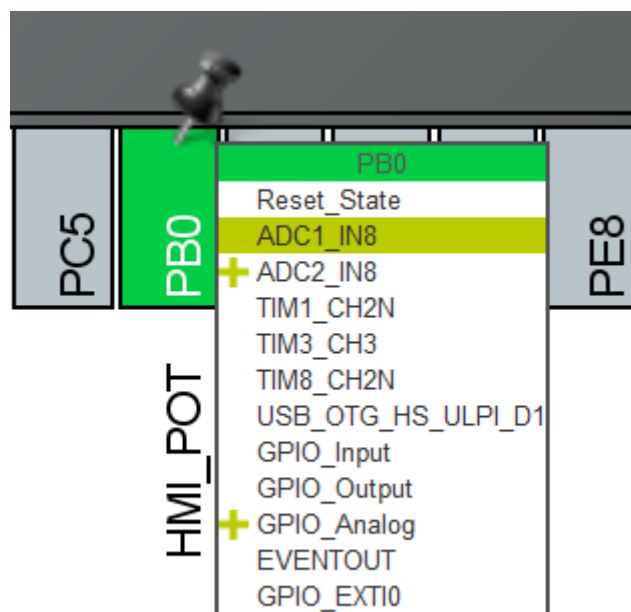
A feladat során a potméter kimenetén lévő feszültségértéket fogjuk megmérni. Ehhez a fejlesztőeszközünk A/D átalakítóját hívjuk segítségül. Az átalakított értéket digitalizált formában és feszültségre átszámított értékben is megjelenítjük majd a kijelzőnkön. Állítsuk össze a következő kapcsolást:



5. ábra: Példakapcsolás felépítése

ADC PIN KIVÁLASZTÁSA A CUBEMX FELÜLETÉN KERESZTÜL

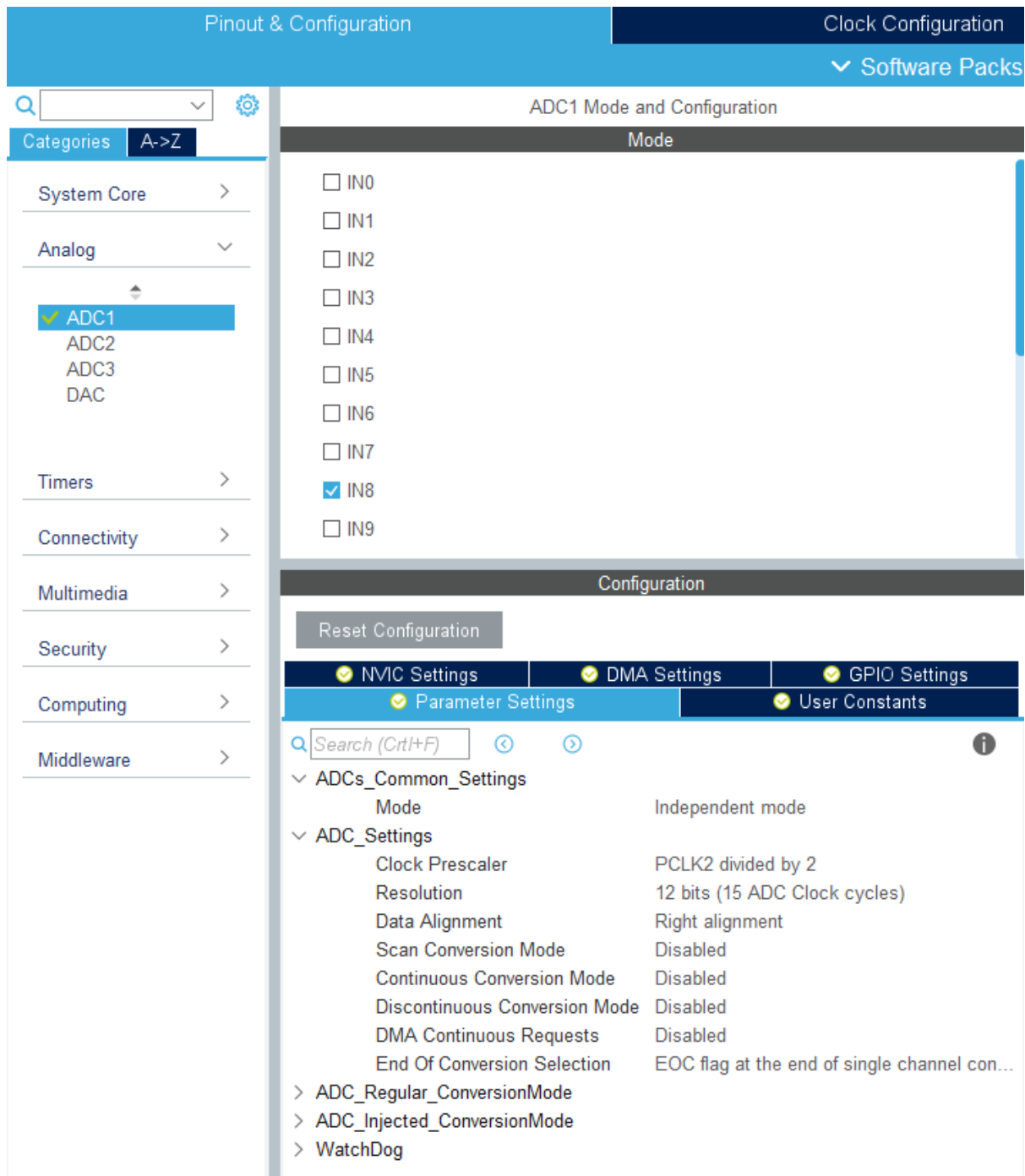
A kapcsolat összeállítását követően nézzük meg, hogyan is történik az ADC periféria kezelése. Először a 4. tananyagrészen már megismert grafikus felületen (CubeMX) bekonfiguráljuk az ADC modulunkat. A potméter kimenetét az áramkörön POT néven szereplő PB0-ás lábra kötöttük, így ezt a lábat analóg bemenetként kell felkonfigurálnunk, majd elneveznünk. A következő képen látható, hogyan tudjuk ezt megtenni.



6. ábra: ADC pin beállítása

ADC PERIFÉRIA BEÁLLÍTÁSA

Ezt követően a bal oldali menüsor Analog menüpontja és azon belül is az ADC1 beállításai alatt már meg tudjuk adni, hogy milyen konfigurációval szeretnénk majd a perifériát használni.



The screenshot shows the STM32CubeMX configuration tool. The left sidebar has a tree view with 'Analog' expanded and 'ADC1' selected. The main area is titled 'ADC1 Mode and Configuration' and is divided into 'Mode' and 'Configuration' sections.

Mode Section:

- Inputs: IN0, IN1, IN2, IN3, IN4, IN5, IN6, IN7, ☒ IN8, IN9

Configuration Section:

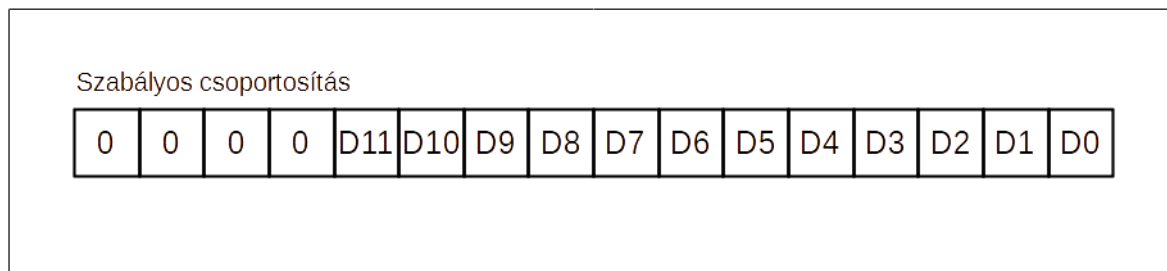
- Buttons: Reset Configuration, NVIC Settings, DMA Settings, GPIO Settings, **Parameter Settings**, User Constants
- Search bar: Search (Ctrl+F)
- Expandable list:
 - ADCs_Common_Settings
 - Mode: Independent mode
 - ADC_Settings
 - Clock Prescaler: PCLK2 divided by 2
 - Resolution: 12 bits (15 ADC Clock cycles)
 - Data Alignment: Right alignment
 - Scan Conversion Mode: Disabled
 - Continuous Conversion Mode: Disabled
 - Discontinuous Conversion Mode: Disabled
 - DMA Continuous Requests: Disabled
 - End Of Conversion Selection: EOC flag at the end of single channel con...
 - ADC_Regular_ConversionMode
 - ADC_Injected_ConversionMode
 - WatchDog

7. ábra: ADC csatorna konfigurációs beállításai

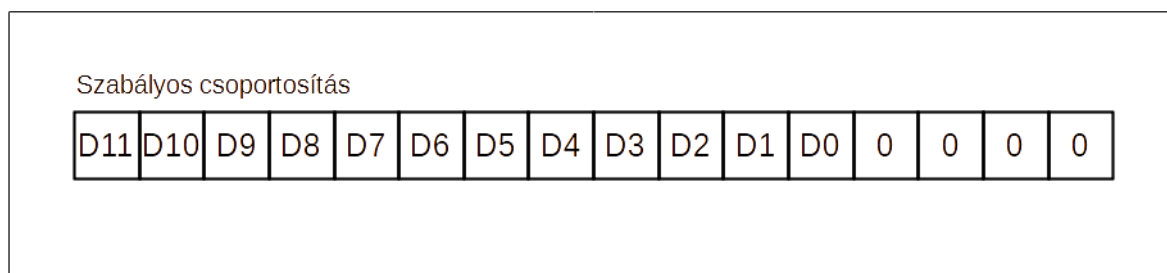
Nézzük meg, milyen paraméterbeállítási lehetőségeink adódnak. Beállíthatjuk az órajel-előosztásunkat (angolul Clock Prescaler), ez a perifériánk órajelét befolyásolja. Az ADC perifériák a PCLK2 órajelet kapják meg, amelynek maximális értéke a fejlesztőkártyán található mikrokontroller esetében 84 MHz lehet (a feladatunk során mi 16 MHz-en használjuk, a beállítást a Clock Configuration menüben tehetjük meg). Ez osztható le 2,4,6 vagy akár 8-adára, mi jelen projektben 2-es osztást

fogunk használni. A következő sorban választható ki, hogy milyen felbontással (angolul Resolution), vagyis hány biten szeretnénk majd a konverziót végrehajtani. Alapértelmezetten 12 bites értékre történik a konverzió, de ez állítható 10, 8 és 6 bites értékre is, ekkor természetesen az átalakítás is gyorsabb lesz, mivel értelemszerűen kevesebb ADC órajelciklus szükséges. Az adat igazítása (angolul Data Alignment) fogja meghatározni, hogy a konverzió eredményét tároló 16 bites regiszterben az eredményt az alsó vagy a felső bitek tartalmazzák.

Jobbra igazított adat



Balra igazított adat



8. ábra: Adatok lehetséges igazítása

Továbbá ki kell választanunk az ADC módját, amely jelen projektben az egyszeri konverziós. A többi, korábban ismertetett opciót kikapcsoljuk. Meg kell adnunk továbbá a konverzió végét jelző eseményt. Ez bekövetkezhet az egyes csatornák átalakításának befejeződésekor, vagy amikor az összes konverzió lezajlott.

CSATORNA BEÁLLÍTÁSA

A következő legördülő menüben ("Regular_ConversionMode - rendszeres konverziós mód) adható meg a használni kívánt csatornák száma, ami jelen esetben 1 (ez már az átalakítás jellégből is következik).

▼ ADC_Regular_ConversionMode	
Number Of Conversion	1
External Trigger Conversion Source	Regular Conversion launched by software
External Trigger Conversion Edge	None
▼ Rank	1
Channel	Channel 8
Sampling Time	3 Cycles

9. ábra: ADC konverziós mód beállításai

Beállítható, hogy a konverziós folyamat egy külső kiváltó (trigger) esemény hatására induljon el vagy pedig előre meghatározott módon, szoftverből kezdeményezzük. Külső megszakítást például a mikrovezérlő hardveres időzítői tudnak generálni, hogy valamely lejártakor induljon el egy konverzió (így spórolva meg néhány műveletet ahhoz képest, mintha ugyanezt szoftveres időzítővel valósítottuk volna meg). Míg az átalakítás szoftveres indítása során a folyamat megkezdése egy függvényhívással fog történni (ennek mikéntjéről kicsit lejjebb tanulunk majd). Több csatorna használatakor, már rangsorba (angolul Rank) kell állítanunk az egyes csatornákat, amely alapján növekvő sorrendben fognak lezajlani a konverziók. Például, ha az 5. és a 8. ADC csatornát szeretnénk használni, akkor beállíthatjuk, hogy először az 5. csatorna (kisebb rangú, nagyobb prioritású) konverziója hajtódjon végre és csak ezt követően a 8. csatornát (nagyobb rangú, kisebb prioritású). A csatornához még meg kell adnunk egy mintavételezési időt is (angolul Sampling Time), amely ADC órajelciklusban értendő. Ez azt jelenti, hogy ennyi ideig fog töltődni az ADC bemeneti puffer kondenzátora, mielőtt megtörténne a mintavétel a jelből. Természetesen minél nagyobb ez a szám, annál pontosabb eredményt kaphatunk (lényegében egy átlagolt értéket), bár ekkor ez több időt is fog majd igénybe venni.

AZ ADC PERIFÉRIA INICIALIZÁLÁSA A PROGRAMUNKBAN

A generált projektben megtalálhatjuk az ADC perifériánk inicializálásáért felelős részt az adc.c fájlban. Nézzük meg ezt kicsit részletesebben!

A HAL könyvtár biztosít számunkra egy ADC_HandlerTypeDef struktúrát, amely segítséget nyújt az ADC periféria egyszerű használatához. Ebben a struktúrában tároljuk el az ADC-hez tartozó konfigurációt, illetve a könyvtár nyújtotta függvények is ezeken végeznek műveletet. Létrehoztunk (generáltunk) egy ADC_HandlerTypeDef típusú globális változót hadc1 néven.

```
ADC_HandlerTypeDef hadc1;
```

Az ADC periféria számára generált konfiguráló kódot az MX_ADC1_Init() alatt láthatjuk, amely tartalmazza azokat a paramétereket, amelyeket megadtunk a grafikus felületen.

```
/* ADC1 inicializálás */  
void MX_ADC1_Init(void)  
{  
    /* Változó létrehozása a későbbi csatorna konfigurációnak */  
    ADC_ChannelConfTypeDef sConfig = {0};  
  
    /* ADC perifériának a konfigurációja */  
    hadc1.Instance = ADC1;  
    /* Órajel előosztása */  
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
```

```
/* Felbontás megadása */
hadc1.Init.Resolution = ADC_RESOLUTION_12B;
/* Szkenneléses mód letiltása */
hadc1.Init.ScanConvMode = DISABLE;
/* Folyamatos mód letiltása */
hadc1.Init.ContinuousConvMode = DISABLE;
/* Szakaszos mód letiltása */
hadc1.Init.DiscontinuousConvMode = DISABLE;
/* Szoftveres konverzió indításának beállítása */
hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
/* Adat bit-ek igazításának kiválasztása */
hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
/* Konverzió során használt csatornák száma */
hadc1.Init.NbrOfConversion = 1;
/* DMA letiltása */
hadc1.Init.DMAContinuousRequests = DISABLE;
/* Konverzió végét jelző esemény kiválasztása */
hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
/* ADC-t inicializáló függvény meghívása */
if (HAL_ADC_Init(&hadc1) != HAL_OK)
{
    /* Hibakezelés */
    Error_Handler();
}
/* Csatorna konfigurációja */
/* Csatorna megadása */
sConfig.Channel = ADC_CHANNEL_8;
/* Rang megadása */
sConfig.Rank = 1;
/* Mintavételezés időtartama a konverzió során */
sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
/* Csatornát inicializáló függvény meghívása */
if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
```



```
{  
    /* Hibakezelés */  
    Error_Handler();  
}  
  
}
```

A `hadc1` struktúránk `Instance` eleme tartalmazza, hogy a mikrovezérlőnk melyik ADC perifériájához fognak tartozni a konfigurációs paraméterek. Jelen esetben ez az `ADC1`, ami lényegében a regiszter címe, amelyen keresztül el tudjuk érni a mikrokontrollerünk első ADC perifériáját. Ezt követően láthatjuk az inicializálandó elemeket, ezek mind az `Init` alatt foglalnak helyet. A `HAL_ADC_Init()` függvény végzi el az ADC periféria konfigurációját, lefutása során bekerül a kívánt konfiguráció a megfelelő regiszterekbe. Ezt megelőzően fizikailag is inicializálásra kerül a csatornához tartozó GPIO láb, amelyre az adott ADC ki van kivezelve, ezt a `HAL_ADC_MspInit()` függvényen belül találjuk.

A használni kívánt csatorna paramétereit egy `ADC_ChannelConfTypeDef` típusú struktúrában adjuk meg. Ez tárolja a csatorna azonosítóját, a rangsorban betöltött helyét és hogy mennyi ideig mintavételezzük a jelet az átalakítás során. Az előbbieken összeállított csatornaconfiguráció a `HAL_ADC_ConfigurationChannel()` függvénynek átadva jut érvényre.

```
void HAL_ADC_MspInit(ADC_HandleTypeDef* adcHandle)  
{  
  
    GPIO_InitTypeDef GPIO_InitStruct = {0};  
    if(adcHandle->Instance==ADC1)  
    {  
        /* ADC1 órajelének engedélyezése */  
        __HAL_RCC_ADC1_CLK_ENABLE();  
  
        __HAL_RCC_GPIOB_CLK_ENABLE();  
        /* GPIO konfigurációja */  
        GPIO_InitStruct.Pin = HMI_POT_Pin;  
        /* ADC mód beállítása */  
        GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;  
        /* Nincs belső szintrehúzás */  
        GPIO_InitStruct.Pull = GPIO_NOPULL;  
    }
```

```
/* GPIO-t inicializáló függvény */  
HAL_GPIO_Init(HMI_POT_GPIO_Port, &GPIO_InitStruct);  
  
}  
}
```

Itt történik meg az ADC és I/O perifériák órajelének engedélyezése. A GPIO struktúrájában meg kell adni a Pin-nél az adott láb azonosítóját, a Mode alatt pedig azt, hogy ADC (ANALOG) módban fogjuk használni. A konfiguráció érvényesítésére a HAL_GPIO_Init() függvényt hívjuk meg az előbbieken összerakott I/O konfigurációval.

Ezáltal inicializáltuk a fizikailag használandó I/O lábat és magát az ADC perifériát is. Ezt követően a mikrovezérlő ADC modulja már használatra kész.

AZ ÁTALAKÍTÁS FOLYAMATA BLOKKOLÓ ÜZEMMÓD ESETÉN

Térjünk át a main.c fájlra és nézzük meg a programunkat! Első példánkban blokkoló üzemmódban használjuk az ADC-eket. Ez a következőképpen fog kinézni.

```
/* 32 bites változó az eredmény eltárolásához */  
uint32_t adc;  
  
/* Konverzió indítása blokkoló módban */  
HAL_ADC_Start(&hadc1);  
/* Ellenőrizzük, hogy a konverzió sikerrel végződött-e? */  
if(HAL_ADC_PollForConversion(&hadc1, 10)==HAL_OK)  
{  
    /* A konverzió eredményének kiolvasása */  
    adc = HAL_ADC_GetValue(&hadc1);  
}
```

A konverzió elindítását blokkoló módban a HAL_ADC_Start() függvénnyel tehetjük meg, amelynek át kell adnunk az előzőleg felkonfigurált ADC perifériát reprezentáló struktúrát.

A HAL_ADC_Start() hibakezelést is tartalmaz, amely során ellenőrzésre kerül például az is, hogy a periféria egyáltalán be van-e kapcsolva, ha nincs, akkor először is bekapcsoljuk az ADC-t. Honnan fogjuk tudni, hogy a konverzió befejeződött? Erre szolgál a HAL_ADC_PollForConversion() függvény, amely paraméterül a korábban használt ADC struktúrát várja, illetve egy időtúllépési (angolul timeout) értéket. Ez azt a maximális időt jelenti ezredmásodpercben, ameddig várakozhatunk a konverzió

végére. Példánkban az átadott érték 10, tehát maximum 10 ezredmásodpercig várunk az átalakítás eredményére. Szemléltetésképpen, ennyi idő alatt a maximális konverziós sebesség mellett (2,8 Msps) 28 000 darab átalakítás történne meg.

Amennyiben a művelet hamarabb befejeződik, mint, hogy elérjük az időkorlátot, akkor egyből visszatérünk a főprogramba. Ha pedig letelt az idő, akkor a függvény kilép hibával és a konverzió sikertelen volt. Az eredményt az ADC_DR regiszterből kapjuk meg, amelyet a HAL_ADC_GetValue() függvény segítségével olvassuk ki, majd pedig a kijelzőn megjelenítjük az értékét. Lehetőségünk van megszakítani a konverziót és kikapcsolni az ADC perifériánkat a HAL_ADC_Stop() függvény segítségével.

Sikeres kódfeltöltés után, a kijelzőn az "ADC value" ("ADC érték") feliratnak kell szerepelnie, ami alatt egy 0 és 4 095 közötti számértéket, majd pedig 0 és 3 300 mV közötti feszültségértéket kell látnunk. A szélsőértékeket értelemszerűen a potméter végállásaiban kapjuk. Ez volt a legegyszerűbb mérési mód, most nézzük meg a második feladatunkat, ahol kicsit ugyan bonyolultabb, de hatékonyabb megoldást kapunk.

KONVERZIÓ FOLYAMATA MEGSZAKÍTÁSOS MÓD ESETÉN

A mikrovezérlőnk jobb kihasználása érdekében az ADC perifériánkat megszakításos módban fogjuk használni. Ez azt jelenti, hogy az átalakítás alatt más műveletek végrehajtása is lehetséges, az eredmény kiolvasása csupán a konverzió befejezését jelző megszakítás után fog megvalósulni. Menjünk vissza először a CubeMX felületére és engedélyezzük a megszakításokat az ADC perifériákhoz! Ezt a megszakításvezérlő beállításai (angolul NVIC Settings - Nested Vector Interrupt Controller Settings) alatt tudjuk megtenni.

✓ NVIC Settings	✓ DMA Settings	✓ GPIO Settings
✓ Parameter Settings	✓ User Constants	
NVIC Interrupt Table	Enabled	Preemption Priority
ADC1, ADC2 and ADC3 global interrupts	✓	0
		Sub Priority
		0

10. ábra: ADC megszakítások engedélyezése

A programunkhoz visszatérve a megszakítás inicializálása a HAL_ADC_MspInit() függvényben történik meg a következőképpen.

```
/* ADC1 megszakítás prioritásának beállítása */
HAL_NVIC_SetPriority(ADC_IRQn, 0, 0);
/* ADC1 megszakítás engedélyezése */
HAL_NVIC_EnableIRQ(ADC_IRQn);
```

Az inicializálás során a megszakításhoz prioritást kell rendelnünk, illetve engedélyeznünk kell azt (összetettebb szoftverek esetén, ahol több megszakítást is használhatunk, ott értelmet nyer a megfelelő prioritási sorrend felállítása ezek között).

Térjünk vissza ismét a main.c fájlhoz! A konverzió elindítását ebben az esetben a HAL_ADC_Start_IT() függvény meghívásával tudjuk megtenni.

```
while (1)
{
    ...
    /* Konverzió újraindítása megszakítással */
    HAL_ADC_Start_IT(&hadc1);
    ...
}
```

Ezen belül kerül engedélyezésre az ADC-hez tartozó megszakítás. A regiszterekben történő bitmanipulációk az előző esethez hasonló módon játszódnak le. A konverzió végén hardveres megszakítást generál az ADC periféria, amely hatására meghívódik az ADC_IRQHandler() és azon belül a HAL_ADC_ConvCpltCallback() függvény, amelyet nekünk kell megírni. Mivel a callback függvény tudja, hogy melyik periféria hívta meg, így az átadott “*hadc” paraméter minden esetben a deklarációban szereplő lokális “hadc” változóra mutat. Ha specifikusan például hadc1-t vagy hadc2-t írunk, akkor csak a neki megfelelő perifériának adná vissza az eredményét, bármelyik másik ADC hívja is meg a callback függvényt.

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    /* Konverzió eredményének kiolvasása */
    adc = HAL_ADC_GetValue(hadc);
}
```

A konverzió eredményét kiolvasó függvényünket azért a callback függvényben helyezzük el, mert minden alkalommal, amikor lefut, biztosak lehetünk benne, hogy egy új beolvasás eredményét kapjuk meg. Ha szükségünk van rá, akkor az A/D átalakító leállításához a HAL_ADC_Stop_IT() függvényt tudjuk meghívni. (A példakódban nem szerepel.)

Kitekintés:

Érdekes lehet, hogyha gyorsan tekerjük a potmétert, akkor a visszaszámolt feszültségérték és a digitális reprezentációja elcsúszik egymástól. Ennek az oka az itt látható kódban keresendő, amely a while ciklus elején helyezkedik el a main.c fájlban.

```
/* Konverzió újraindítása megszakítással */
HAL_ADC_Start_IT(&hadc1);

/* Kijelző kezelése */
```

```
sprintf(adc_char, "%d", (uint16_t)adc);
```

Ezt a while(1) ciklus végén található HAL_Delay() függvény paraméterének magasabb értékre állításával könnyebben lehet látni. Mivel megszakításos módban használjuk a perifériát, így a konverzió elindítása után a program továbblép a kiíratáshoz, ahol az "adc" változóban még a korábban átalakított érték digitális reprezentációja szerepel, mivel még nem fejeződött be az átalakítás. A megoldás az, hogy a program későbbi részében hívjuk meg a kiíratást végző függvényt, például áthelyezzük a második kurzor beállító és az értéket kiíró utasítások közé, ahogy az az alábbi sorokban látható:

```
OLED_Fill(Black);
```

```
OLED_SetCursor(2, 0);
```

```
OLED_WriteString("ADC value", Font_11x18, White);
```

```
OLED_SetCursor(2, 18);
```

```
/* Kijelző kezelése */
```

```
sprintf(adc_char, "%d", (uint16_t)adc);
```

```
OLED_WriteString(adc_char, Font_11x18, White);
```

```
sprintf(adc_char, "%dmV", (uint16_t)(adc*0.806));
```

```
OLED_SetCursor(2, 18+18);
```

```
OLED_WriteString(adc_char, Font_11x18, White);
```

```
OLED_UpdateScreen();
```

Ha ezt megtesszük, akkor mire felhasználnánk az "adc" változót, addigra már a helyes érték fog benne szerepelni.

JEL STABILITÁSÁNAK JAVÍTÁSA

Megfigyelhetjük, hogy hiába engedjük el a potmétert, a kijelzőn megjelenített érték nem stabilan egy állandó értéket mutat, hanem kissé ugrál. Ez a pontatlanság az ADC belső hibájából, illetve a mérési zajból származhat. Ennek kiküszöbölése érdekében állítsuk az ADC csatorna beállításánál a mintavételezési időt 480 órajelciklusra, ezt a CubeMX eszközbeállító felületén végezzük el, majd mentést követően generáljuk újra a forráskódot! Az adc.c-ben a következő sort fogjuk látni.

```
// Mintavételezési idő beállítása
```

```
sConfig.SamplingTime = ADC_SAMPLETIME_480CYCLES;
```

Így lényegében a jel egymást követő 480 időpillanatának az átlagát fogjuk kapni. Ugrálás ugyan most is látható, de mérséklődött (az én esetemben az amplitúdó ötödére csökkent). Nézzünk meg egy

szoftveres átlagolást, számoljunk mozgóátlagot! Ez azt jelenti, hogy az aktuális eredményünket mindig a legutóbbi és az öt megelőző meghatározott számú mért értékből képzett átlag fogja adni. Példánkban az átlagot 25 egymást követő konvertált értékből számoljuk ki. A számításhoz szükségünk van egy tömbre, egy, a mozgóátlagot tartalmazó változóra és egy mutatóra, amellyel számontartjuk, hogy hová helyeztük el a tömbben az utolsó konverzió eredményét. Ezt a mutatót minden egyes konverzió után növeljük. Pontosan 25 konverzióval később fogunk visszatérni a tömb legelső elemére, így fog megvalósulni az, hogy mindig csak a legfrissebb 25 értéket tartalmazza a tömbünk.

```
/* Mozgóátlag számításhoz szükséges változók, globálisan létrehozva */  
/* A 25 egymást követő értéket tároló tömb */  
uint32_t adc_avg[25] = {};  
/* Az utoljára betett érték helyét tartalmazó mutató */  
uint8_t p_avg_cnt = 0;  
/* A kiszámolt átlagot tartalmazó változó */  
uint32_t avg = 0;
```

A mozgóátlag-számítás megvalósítása a következőképpen néz ki:

```
/* Ellenőrizzük, hogy a konverzió befejeződött-e */  
if(HAL_ADC_PollForConversion(&hadc1, 10)==HAL_OK)  
{  
    /* A konverzió eredményének kiolvasása */  
    adc = HAL_ADC_GetValue(&hadc1);  
    /* Megvizsgáljuk, hogy elértünk-e már a tömb végére */  
    if(p_avg_cnt == 25)  
    {  
        /* Tömb első elemére állítjuk a mutatót */  
        p_avg_cnt = 0;  
    }  
    /* Elhelyezzük a tömbben az utolsó konverzió eredményét */  
    adc_avg[p_avg_cnt] = adc;  
    /* Növeljük a mutató értékét */  
    p_avg_cnt++;  
    avg = 0;  
    /* A 25 elemből meghatározzuk az átlagot */
```

```
    for(uint8_t i = 0; i<25; i++)
    {
        /* Tömb elemeit összeadjuk */
        avg += adc_avg[i];
    }
/* Az átlag kiszámolásához az előző összeget osztanunk kell az elemszámmal */
    avg = avg/25;
}
```

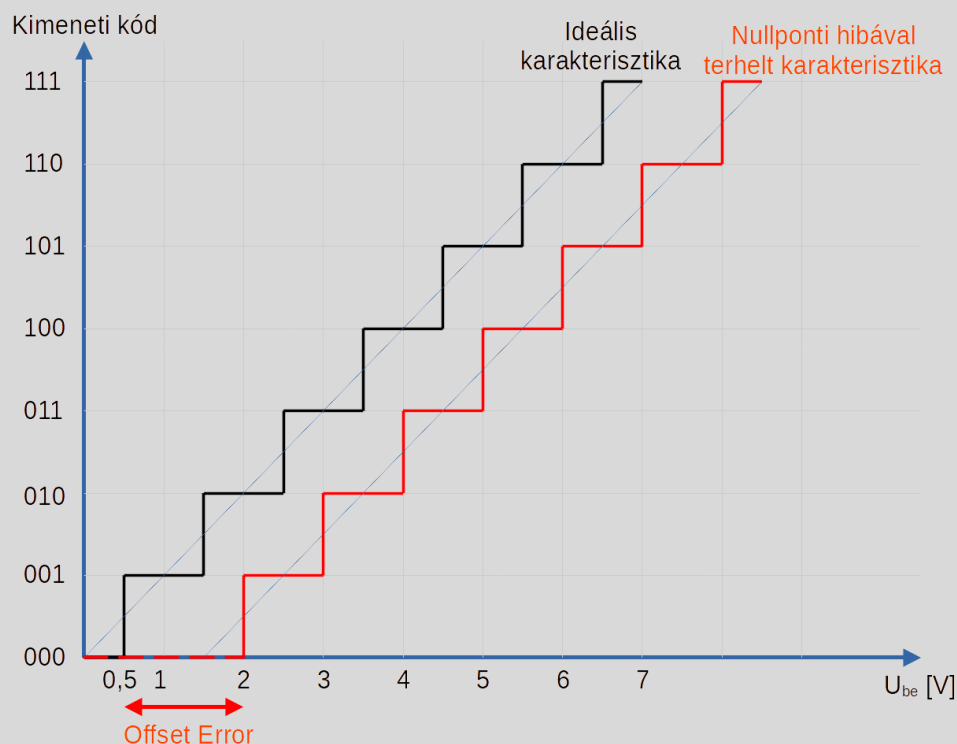
Látható, hogy így egy stabilabb (lassabban változó) értéket kapunk a kijelzőn. Ugyanakkor megfigyelhetjük, hogy a potméter tekerésekor az új érték az előző példákhoz képest sokkal lassabban változik meg. Ez amiatt van így, mert az átlagolással a változást később érzékeljük. Ezzel a mérés pontosságát nem rontottuk, de sikeresen csökkentettük a kilengések nagyságát (igaz, a sebesség rovására).

Kitekintés:

ADC perifériánk pontosságát számos hiba befolyásolja, ezeket két csoportba sorolhatjuk. Az egyik csoportba azok a hibák tartoznak, amelyek magától az ADC modultól származnak, a másikba pedig azok, amelyek a környezetből jönnek. Nézzük is az első csoportot, azon belül pedig a nullponthibát (angolul offset error).

Ehhez nézzünk egy szemléletes példát. Vegyünk egy 3 bites ADC-t, ekkor összesen $2^3 = 8$ különböző értéket tudunk megjeleníteni. A bemeneti feszültségtartomány pedig legyen 0 - 7 V, ekkor egy LSB pontosan 1 V-nak felel meg.

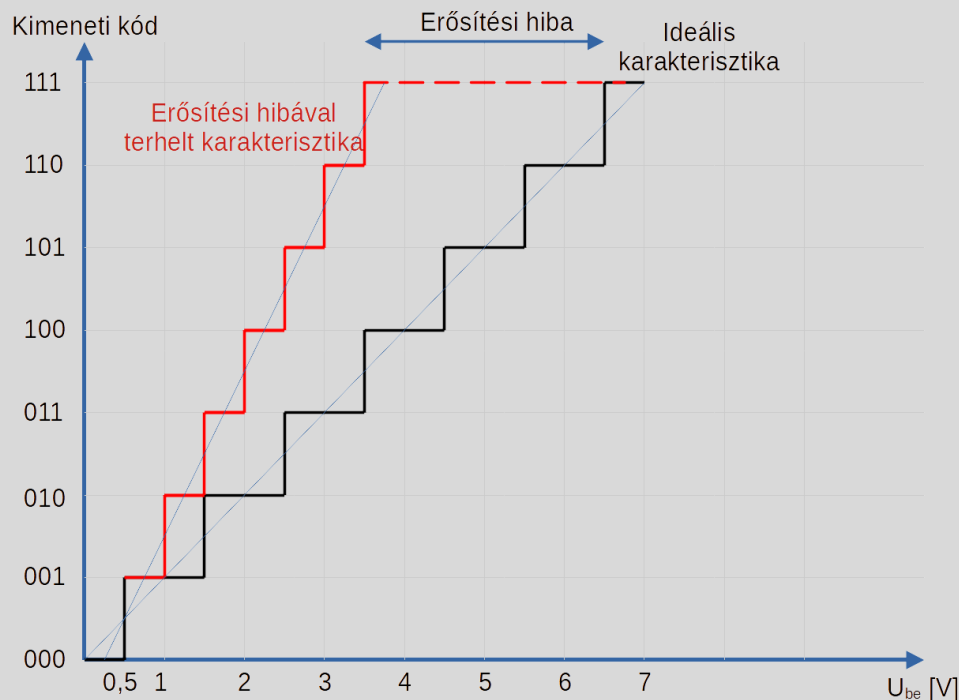
Ideális esetben az A/D átalakító kimenetén az első átmenet 0,5 LSB-nél, azaz jelen esetben 0,5 V-nál következik be. Offset error esetén a váltás ettől eltérő értéken fog megtörténni, mint például az alábbi ábrán.



11. ábra: Offset hiba

Látható, hogy ebben az esetben a digitális érték első bitjének megváltozása az ideálisnál jóval később, 2 V-nál történik meg, azaz a nullponthiba nagysága $2\text{ V} - 0,5\text{ V} = 1,5\text{ V}$. Ez természetesen azt is jelenti, hogy a karakterisztika többi átmenete is eltolódik.

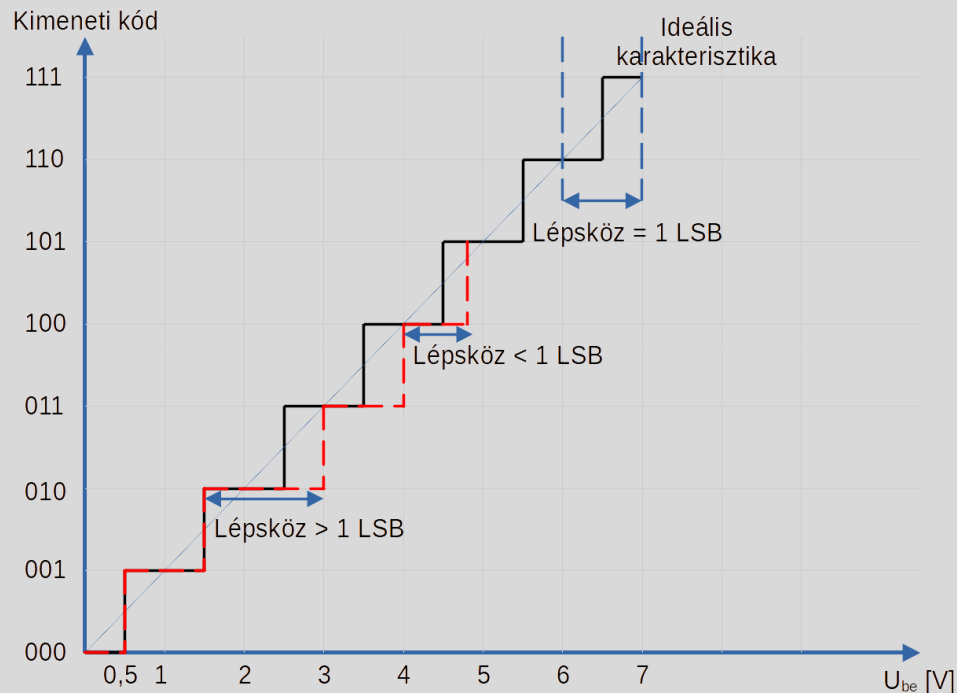
A következő hiba, amit érdemes megismernünk, az az erősítési hiba (angolul gain error). Ennek a hibának a nagyságát a karakterisztika utolsó átmeneténél megmérjük. Az eltérést az okozza, hogy az átviteli függvény meredeksége különbözik az ideálistól. Tekintsük az alábbi ábrát.



12. ábra: Erősítési hiba

Könnyedén kiszámolhatjuk, hogy az ideális karakterisztika meredeksége 1 V/lépcsőfok, míg a valós esetben ez az érték 0,5 V/lépcsőfok. Számítsuk ki tehát az erősítési hibát! Ennek első lépése az offset error eliminálása, tehát fogjuk a valós karakterisztikát és az első átmenet helyét az ideális pontra toljuk. Második lépésben pedig a maximális digitális kódnál (jelen esetben ez az 111) kiszámoljuk a valós és az ideális karakterisztika által kapott értékek különbségét (azaz az erősítési hibát), ami jelen esetben 3 LSB. A példánkban felvázolt hibák pozitív előjellel szerepelnek, de természetesen fordított előjellel is előfordulhatnak.

A karakterisztika egy másik jellegű hibája a differenciális linearitási hiba (angolul differential linearity error). Ekkor egy-egy lépésköznyi távolság szélesebb, vagy épp keskenyebb lesz az ideális 1 LSB-nyi távolságnál. A következő ábrán megfigyelhetjük, hogy néhol hosszabb, illetve néhol rövidebb egy lépésköz az ideálishoz képest.

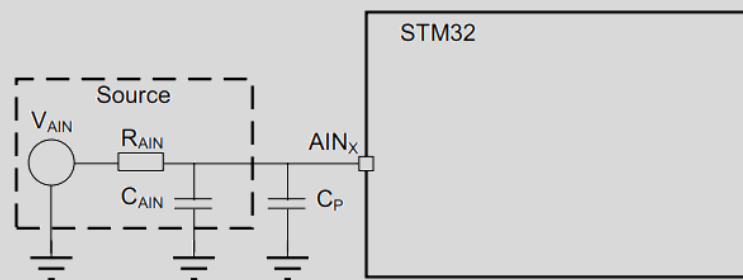


13. ábra: Differenciális nemlinearitási hiba

A másik csoportba tartozó hibák magától a környezettől származtathatóak. A legtöbb hiba a referenciafeszültséghez kapcsolható. Mivel ez az átalakításunk alapja, ezért a névleges értéktől való néhány százalékos eltérése is számottevő hibához vezethet a konverzió pontosságát illetően. A referenciafeszültség pontatlansága következhet az ezt előállító áramkör hibájából vagy a környezeti zajok által is. Ennek hatására akár azt is láthatjuk, hogy a kapott digitalizált értékünk nem stabil, akár több LSB-nyi ugrálást is felfedezhetünk.

Előfordulhat, hogy a referenciafeszültségünk hőmérsékletfüggést mutat, ekkor melegedés hatására kismértékben változhat a feszültségérték, de ez a változás is LSB-kkel képes a mérésünket torzítani.

A mérni kívánt analóg jelre is sok különböző zaj rakódhat rá, amelyek által teljesen pontos átalakítás esetén sem azt a digitális értéket kapjuk, amelyre elméletileg számítanánk. Zajforrások lehetnek például közelben folyó áramok, vagy akár más elektronikai eszközök is. Továbbá az ADC bemenete és a jelforrás között (vagy magában a jelforrásban) előforduló kapacitás vagy ellenállás (ahogyan ez a következő ábrán látható) növelni fogja az ADC belső kapacitásának feltöltési idejét, amely ha nagyobb lesz, mint a mintavétel ideje, akkor következőnek a még töltődő belső kapacitás feszültségét mérjük, így pontatlan értéket kaphatunk.



14. ábra: Nem megfelelő kapcsolásból, elrendezésből adódó hiba

Forrás: STMicroelectronics

https://www.st.com/resource/en/application_note/cd00211314-how-to-get-the-best-adc-accuracy-in-stm32-microcontrollers-stmicroelectronics.pdf

Természetesen ezekből a hibákból származó pontatlanságokat tudjuk csökkenteni. Erre többféle módszer is kínálkozik, például a gondosabb hardvertervezés. Használhatunk külső zajszűrőt is, amivel a bemenő jelre rakódó nagyobb frekvenciás zajokat lehet szűrni még mielőtt a jel eljutna az AD átalakító bemenetére. Egy könnyedén megvalósítható megoldás, amit mi is alkalmaztunk, az átlagolás, így a kiugró, pontatlan értékeket úgymond el tudjuk simítani, ezáltal pontosabb mérési eredményt kapunk.

ÖSSZEFOGLALÁS

Ebben a tananyagrészben megismerkedtünk az ADC perifériával, először nagy vonalakban annak jellemzőivel, majd pedig mélyebben a fejlesztőpanelen található STM32-es mikrovezérlőbe integrált ADC-vel. A periféria beállítását a STM32CubeIDE grafikus felület segítségével hajtottuk végre. Ezt követően megnéztük az A/D konverter inicializálását a szoftverünkben, majd áttanulmányoztuk a blokkoló és a megszakításos üzemmódok működését. Végezetül pedig egy átlagolási módszerrel javítottunk a mérés pontosságán.